AgentGate

Day 2 Short Note: Validation Errors, Unit Tests, Cache

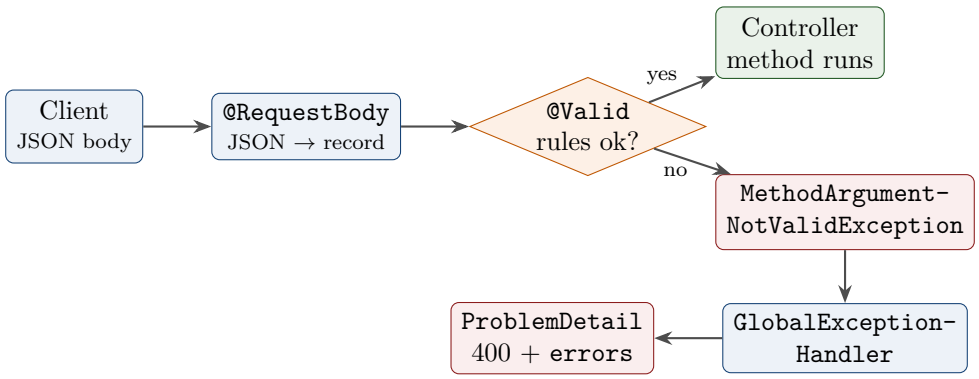
Tuesday, 27 October 2026 | Week 1 finished

Summary

Done today	`GlobalExceptionHandler`, 5 Mockito unit tests, Caffeine cache with eviction.
Result	Week 1 is complete, five days before the 1 November deadline.
Proof	Invalid request returns a clean `application/problem+json` 400. `Tests run: 5, Failures: 0`. Cache measured: repeated checks add no database query, and a new rule clears the cache.
Next	Week 2: Auth Service (OAuth2, token exchange) and Gateway. Starts 2 November.

1 Validation errors: `@Valid` and the exception handler

`@Valid` means: *before my method runs, check the incoming object against the rules written on its fields.* The rules (`@NotBlank`, `@NotNull`) do nothing without it.




```
@RestControllerAdvice                          // watches ALL controllers
public class GlobalExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)   // handles this exception
    public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
          .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
        ProblemDetail p = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST,
                                                           "One or more fields are
    invalid");
        p.setTitle("Validation failed");
        p.setProperty("errors", errors);
        return p;
    }
}
```

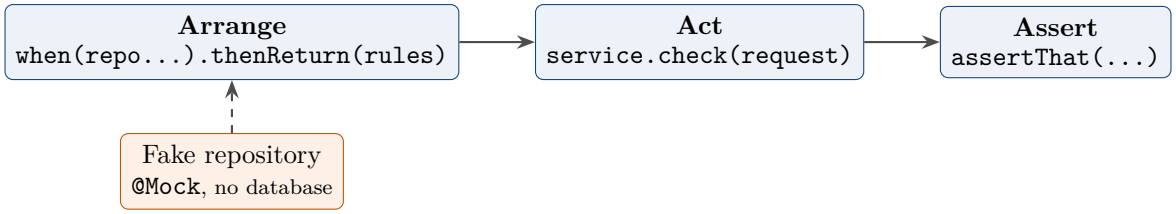
Real response (blank `tenantId` and `agentId`, no `effect`):

```
{ "type":"about:blank", "title":"Validation failed", "status":400,
  "detail":"One or more fields are invalid", "instance":"/policies",
  "errors": { "effect":"must not be null",
              "tenantId":"must not be blank", "agentId":"must not be blank" } }
```

`HashMap` vs `LinkedHashMap`: both give O(1) lookups. `HashMap` has no order guarantee; `LinkedHashMap` keeps *insertion order* for a small memory cost. We use it so the `errors` output is predictable.

2 Unit tests with Mockito

We test the *thinking* of the service (the allow/deny rules), not the database. A `@Mock` repository is a fake that returns exactly the rules we choose. This works without Spring because of constructor injection.



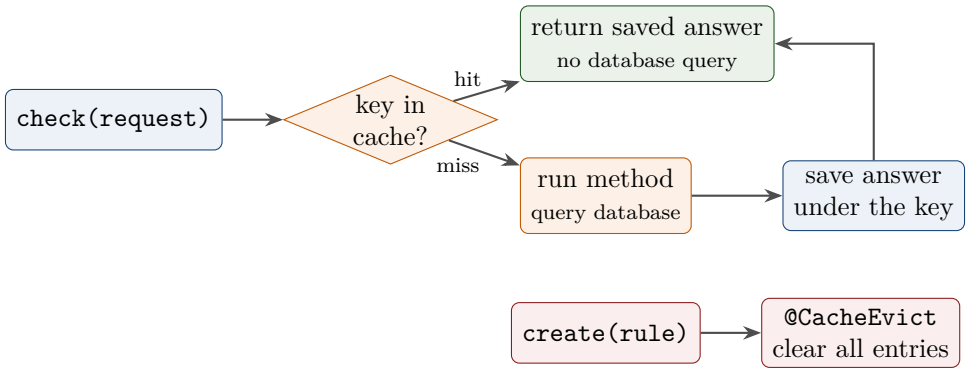
Test	Rules the fake returns	allowed	reason
`noRules_isDeniedByDefault`	none	false	no matching rule
`onlyAllowRule_isAllowed`	ALLOW	true	allowed by policy
`onlyDenyRule_isDenied`	DENY	false	denied by policy
`allowThenDeny_denyWins`	ALLOW, DENY	false	denied by policy
`denyThenAllow_denyWins`	DENY, ALLOW	false	denied by policy

Lessons

- A method without `@Test` is *silently skipped*. Two tests were green but never ran. Always check the count: `Tests run: 5`.
- The reversed-order test proves the result does not depend on the order of rows. It would catch the old "return on the first row" bug.
- You cannot write `new PolicyRepository()`: it is an interface with about 40 inherited methods. `@Mock` is the short way to get a fake.

3 Caffeine cache

`check` runs on every agent action, so repeated questions are answered from memory. The cache is **not** Redis: it lives inside the Java process (fastest, no network), and Redis is kept for the Budget Service and the Gateway rate limit.




```
@Cacheable("policyChecks")                                  // on check()
@CacheEvict(value = "policyChecks", allEntries = true)      // on create()
```

Cache name	`"policyChecks"` is a label for the *whole* cache (a map with many entries), not one entry.
Key	One parameter ⇒ the parameter itself (the `CheckRequest` record). Records compare by value, so equal requests share one entry.
Value	What the method returned (`CheckResponse`).
Shared?	By everyone who calls this app instance. The key is the *question*, not the caller. `tenantId` is in the key, so tenants never mix.
`allEntries`	Clears every entry. Needed because `create` receives a `CreatePolicyRequest`, which never matches a `CheckRequest` key.
Limits	`maximumSize=1000`, `expireAfterWrite=60s` (safety net).

Measured with SQL logging (number of `SELECT` queries on `policies`):

Step	Result	Total SELECTs
1. first check	false (miss)	1
2. same check	false (hit)	1
3. same check	false (hit)	1
create an ALLOW rule	cache cleared	
4. check again	**true** (miss)	2
5. same check	true (hit)	2

Limits to remember

- Each running copy has its *own* cache. A `create` on copy 1 does not clear copy 2, so copy 2 can be stale for up to 60 s. Fix later with Redis or Kafka invalidation.
- If 100 requests miss the same new key at once, all may query the database (stampede). `sync = true` makes one fetch while the rest wait.
- Like `@Transactional`, it works through a proxy: calling the method from the same class skips the cache.
- The same cache name must be used in `@Cacheable` and `@CacheEvict`; a typo leaves stale answers.

4 `@Transactional(readOnly = true)`

Used on `check` and `list`, which only read.

Benefit	Why
Faster	Hibernate skips the dirty check and flush at the end.
Safer	The database connection is read only, so an accidental write is rejected.
Consistent	All reads in the method see one view of the data.
Clear	The reader sees at once that the method changes nothing.

`create` writes, so it uses plain `@Transactional`. It is a hint, not a lock.

5 DAO and git

- **DAO** (Data Access Object) hides database code behind an interface. In Spring Data, a *repository is the DAO*, and Spring writes the implementation. A hand-written `dao/` class is for custom access, like the Redis Lua script in Week 3.
- `CLAUDE.md` is no longer tracked: `git rm -cached CLAUDE.md` plus a `.gitignore` entry. The file stays on disk. The first (already pushed) commit still contains it in history.

6 Week 1 checklist

Done: table, entity, repository, DTOs, service, controller

Done: clean validation errors (`ProblemDetail`)

Done: 5 unit tests pass

Done: Caffeine cache with eviction, measured

Left: commit, push, and the 3-line daily note

Interview answers from today

1. *How do you handle validation errors?* "`@Valid` checks the DTO before the controller runs. A `@RestControllerAdvice` turns the exception into a standard `ProblemDetail` 400 that lists each bad field."
2. *How do you test the service without a database?* "Constructor injection lets me pass a Mockito fake repository, so I test the allow/deny logic alone, including that DENY wins in either order."
3. *How does your cache stay correct?* "Policy checks are cached by request in Caffeine. Creating a rule clears the cache, and entries expire after 60 seconds. With several instances I would use Redis or Kafka invalidation."